

Python for Senior Data Platform Engineers

Practical Review Guide for Pair-Programming & Technical Interviews

This guide focuses on the Python knowledge that is most useful for a **Senior Data Engineer / Data Platform Engineer**. The goal is not to memorize every language feature, but to be able to write Python that is:

- correct,
- readable,
- testable,
- memory-efficient,
- extensible,
- maintainable,
- and suitable for production data systems.

For interview preparation, I would study the topics roughly in the order below.

1. Python Data Model & Core Types

Before advanced patterns, you should be extremely comfortable choosing the right built-in data structure.

1.1 List

Ordered, mutable collection.

```
events = [  
    {"id": 1, "type": "created"},  
    {"id": 2, "type": "updated"},  
]  
  
events.append({"id": 3, "type": "deleted"})
```

Typical Data Engineering use:

- storing records
- transformations
- batch results

Access by index is approximately $O(1)$, while searching is $O(n)$.

1.2 Tuple

Ordered but immutable.

```
partition = ("2026", "08", "25")
```

Useful when a value should conceptually not change.

Tuples can also be dictionary keys when their contents are hashable.

```
counts = {  
    ("DE", "purchase"): 120,  
    ("FR", "purchase"): 80,  
}
```

1.3 Dictionary

Probably the **most important Python structure for Data Engineering interviews**.

```
latest_event = {}  
  
for event in events:  
    latest_event[event["id"]] = event
```

Common uses:

- lookup tables
- deduplication
- aggregation
- configuration
- state tracking
- joins

Lookup is approximately:

```
O(1)
```

This is why many DE coding problems eventually reduce to:

```
key → dictionary → state
```

1.4 Set

Stores unique values.

```
processed_ids = set()

if event["id"] not in processed_ids:
    processed_ids.add(event["id"])
```

Excellent for:

- deduplication
- membership checks
- detecting duplicates

Membership is approximately $O(1)$.

1.5 collections

Know these especially well.

defaultdict

```
from collections import defaultdict

revenue = defaultdict(float)

for event in events:
    revenue[event["customer_id"]] += event["amount"]
```

Much cleaner than repeatedly checking whether the key exists.

Counter

```
from collections import Counter

types = Counter(event["type"] for event in events)
```

Result might be:

```
{
    "purchase": 120,
```

```
    "refund": 15,  
}
```

deque

Efficient queue.

```
from collections import deque  
  
queue = deque()  
  
queue.append(event)  
event = queue.popleft()
```

Useful for:

- buffering
- queues
- sliding windows
- BFS-style processing

2. Functions and Clean Function Design

Senior Python code should consist of **small functions with explicit responsibilities**.

Bad:

```
def process(data):  
    # parse  
    # validate  
    # transform  
    # write  
    # log  
    # retry  
    ...
```

Better:

```
def parse(record):  
    ...  
  
def validate(record):  
    ...
```

```
def transform(record):  
    ...  
  
def write(records):  
    ...
```

Then:

```
def process(record):  
    parsed = parse(record)  
    validate(parsed)  
    return transform(parsed)
```

This improves:

- testing
- reuse
- readability
- failure isolation

3. `*args` and `**kwargs`

You should understand them, although you should not overuse them.

```
def log_event(event, **metadata):  
    print(event, metadata)  
  
log_event(  
    "pipeline_failed",  
    pipeline="orders",  
    region="eu",  
)
```

`**kwargs` becomes:

```
{  
    "pipeline": "orders",  
    "region": "eu",  
}
```

Useful for flexible APIs, but excessive use reduces type safety and discoverability.

4. Comprehensions

Pythonic and common in interviews.

Instead of:

```
result = []

for value in values:
    if value > 0:
        result.append(value * 2)
```

Use:

```
result = [
    value * 2
    for value in values
    if value > 0
]
```

Dictionary comprehension:

```
users_by_id = {
    user["id"]: user
    for user in users
}
```

This is especially useful when implementing an in-memory join.

Be careful not to produce unreadable one-liners.

Senior Python code prioritizes readability over cleverness.

5. Sorting and **key**

Frequently useful in DE coding problems.

```
events.sort(key=lambda x: x["timestamp"])
```

Descending:

```
events.sort(  
    key=lambda x: x["timestamp"],  
    reverse=True,  
)
```

Multiple fields:

```
events.sort(  
    key=lambda x: (  
        x["customer_id"],  
        x["timestamp"],  
    )  
)
```

Know that general sorting costs approximately:

$O(n \log n)$

So if you can solve a problem using a dictionary in $O(n)$, that may be preferable.

6. Iterators and Generators

This is one of the most important advanced concepts for Data Engineers.

Consider:

```
def load_records():  
    records = []  
  
    for line in huge_file:  
        records.append(parse(line))  
  
    return records
```

The entire dataset remains in memory.

Instead:

```
def load_records():  
    for line in huge_file:  
        yield parse(line)
```

Now:

```
for record in load_records():  
    process(record)
```

Records are produced lazily.

Conceptually:

```
List  
500M records  
↓  
All loaded into memory  
  
Generator  
record  
↓  
record  
↓  
record  
↓  
processed incrementally
```

Know the difference between:

```
return
```

and:

```
yield
```

A generator is useful when dealing with:

- large files
- streaming data
- API pagination
- database result sets
- memory-sensitive transformations

7. Iterable vs Iterator

An **iterable** can produce an iterator.

Examples:

```
list
tuple
set
dict
```

An **iterator** implements the iteration protocol.

```
iterator = iter([1, 2, 3])

next(iterator)
next(iterator)
```

Result:

```
1
2
```

Generators are iterators.

You do not normally need to implement this manually, but understanding it helps explain streaming processing.

8. Context Managers

Extremely useful for safe resource handling.

Instead of:

```
file = open("events.json")
data = file.read()
file.close()
```

Use:

```
with open("events.json") as file:  
    data = file.read()
```

The resource is closed even if an exception occurs.

Conceptually:

```
Acquire resource  
  ↓  
  use it  
  ↓  
Release resource
```

Common resources:

- files
- database connections
- transactions
- locks
- temporary resources

Custom context manager:

```
from contextlib import contextmanager  
  
@contextmanager  
def transaction(connection):  
    try:  
        yield  
        connection.commit()  
    except Exception:  
        connection.rollback()  
        raise
```

9. Exception Handling

Never write this:

```
try:  
    process()
```

```
except:  
    pass
```

It silently hides failures.

Better:

```
try:  
    process()  
except ValueError as exc:  
    logger.error("Invalid record: %s", exc)  
    raise
```

For a data platform, custom exceptions can make failure types explicit.

```
class PipelineError(Exception):  
    pass  
  
class SchemaValidationError(PipelineError):  
    pass  
  
class SourceUnavailableError(PipelineError):  
    pass
```

Then:

```
if "customer_id" not in record:  
    raise SchemaValidationError(  
        "Missing customer_id"  
    )
```

A good hierarchy lets the caller decide:

```
Schema problem  
→ reject record  
  
Temporary API problem  
→ retry  
  
Unknown system failure  
→ fail pipeline + alert
```

That is much more production-oriented.

10. Type Hints

Modern production Python should generally use type annotations.

```
def calculate_total(  
    values: list[float],  
    ) -> float:  
    return sum(values)
```

For records:

```
def validate(  
    record: dict[str, object],  
    ) -> bool:  
    ...
```

Type hints improve:

- IDE support
- maintainability
- static analysis
- API understanding
- refactoring safety

But remember:

Type hints do not normally enforce types at runtime.

11. Optional and None

Modern syntax:

```
def find_user(user_id: str) -> dict | None:  
    ...
```

Then handle it explicitly.

```
user = find_user("123")
```

```
if user is None:  
    ...
```

Do not confuse:

```
if not user:
```

with:

```
if user is None:
```

because `not user` is also true for:

```
{}  
[]  
""  
0
```

12. Dataclasses

Very useful for structured configuration and domain objects.

Instead of:

```
config = {  
    "source": "orders",  
    "format": "json",  
    "batch_size": 1000,  
}
```

You can use:

```
from dataclasses import dataclass

@dataclass
class PipelineConfig:
    source: str
    format: str
    batch_size: int = 1000
```

Then:

```
config = PipelineConfig(
    source="orders",
    format="json",
)
```

Benefits:

- clearer contract
- typing
- defaults
- automatically generated `__init__`
- easier testing

For immutable configuration:

```
@dataclass(frozen=True)
class PipelineConfig:
    source: str
    batch_size: int
```

13. OOP Fundamentals

For Data Platform Engineering, understand OOP but **do not force everything into classes**.

Use classes when you have:

- behavior + state
 - multiple implementations
 - lifecycle management
 - dependency boundaries
-

14. Encapsulation

Keep internal details hidden.

```
class Pipeline:
    def __init__(self):
        self._processed_count = 0

    def process(self, record):
        self._processed_count += 1
```

The `_` is a convention meaning:

internal implementation detail.

15. Composition Over Inheritance

This is important.

Instead of building huge inheritance hierarchies:

```
Pipeline
  ↓
AwsPipeline
  ↓
S3Pipeline
  ↓
JsonS3Pipeline
```

prefer composing components:

```
Pipeline
├── Source
├── Parser
├── Validator
└── Writer
```

Example:

```
class Pipeline:
    def __init__(
        self,
        source,
```

```
        parser,  
        validator,  
        writer,  
    ):  
        self.source = source  
        self.parser = parser  
        self.validator = validator  
        self.writer = writer
```

This is highly relevant to configurable ingestion frameworks.

16. Abstract Base Classes

Python can define explicit interfaces using [ABC](#).

```
from abc import ABC, abstractmethod  
  
class Source(ABC):  
  
    @abstractmethod  
    def read(self):  
        pass
```

Implementations:

```
class S3Source(Source):  
  
    def read(self):  
        ...
```

and:

```
class ApiSource(Source):  
  
    def read(self):  
        ...
```

The pipeline depends on the abstraction:

```
class Pipeline:  
    def __init__(self, source: Source):
```

```
self.source = source
```

17. Protocol

For modern Python architecture, `Protocol` is particularly valuable.

```
from typing import Protocol

class Source(Protocol):

    def read(self):
        ...
```

Then:

```
class S3Source:

    def read(self):
        ...
```

`S3Source` does not need to explicitly inherit from `Source`.

This follows structural typing:

If it behaves like a `Source`, it can be treated as a `Source`.

For platform frameworks this can reduce unnecessary coupling.

ABC vs Protocol

Think:

```
ABC
"You must inherit from this abstraction."

Protocol
"You must provide this behaviour."
```

Neither is universally superior.

18. Dependency Injection

Very important concept for testable platform code.

Bad:

```
class Pipeline:

    def run(self):
        source = S3Source()
        records = source.read()
```

`Pipeline` creates its own dependency.

Testing becomes harder.

Better:

```
class Pipeline:

    def __init__(self, source):
        self.source = source

    def run(self):
        return self.source.read()
```

Production:

```
pipeline = Pipeline(S3Source())
```

Test:

```
pipeline = Pipeline(FakeSource())
```

This is dependency injection.

The important principle is:

Dependencies should be provided rather than hidden inside business logic.

19. SOLID Principles

You do not need to recite SOLID mechanically, but you should understand the architectural ideas.

S — Single Responsibility

Bad:

```
Pipeline
├─ reads Kafka
├─ parses JSON
├─ validates schema
├─ writes BigQuery
├─ sends Slack alert
└─ handles retries
```

Better:

```
Source
Parser
Validator
Writer
Notifier
```

Each component owns one concern.

O — Open/Closed Principle

Software should be easy to extend without modifying stable core logic.

Bad:

```
if source == "s3":
    ...
elif source == "kafka":
    ...
elif source == "api":
    ...
```

Better:

```
class Source(Protocol):
    def read(self):
        ...
```

Add:

```
class KafkaSource:
```

```
...
```

without modifying the pipeline.

L — Liskov Substitution

Implementations should behave according to the abstraction's contract.

If:

```
Source.read()
```

means:

```
return records
```

then one implementation should not unexpectedly:

```
terminate the process or delete the input data.
```

I — Interface Segregation

Prefer small interfaces.

Bad:

```
class DataPlatform:
    def read()
    def write()
    def validate()
    def alert()
    def deploy()
    def delete()
```

Better:

```
Readable
Writable
Validator
Notifier
```

D — Dependency Inversion

High-level logic depends on abstraction rather than implementation.

```
Pipeline
  ↓
Source
 /   \
S3    API
```

Not:

```
Pipeline
  ↓
S3Source
```

20. Decorators

Decorators wrap behavior around functions.

Example:

```
def log_execution(func):
    def wrapper(*args, **kwargs):
        print(f"Running {func.__name__}")
        return func(*args, **kwargs)

    return wrapper
```

Usage:

```
@log_execution
def ingest():
    ...
```

Useful platform examples:

```
@retry
@metrics
@trace
@validate_config
@authorize
```

But avoid building complicated decorator magic that makes control flow difficult to understand.

21. Design Pattern: Strategy

One of the most relevant patterns for Data Platform Engineering.

Suppose transformation strategy varies.

```
class TransformStrategy(Protocol):  
    def transform(self, record):  
        ...
```

Implementations:

```
class JsonTransform:  
    def transform(self, record):  
        ...
```

```
class AvroTransform:  
    def transform(self, record):  
        ...
```

Pipeline:

```
class Pipeline:  
    def __init__(self, transformer):  
        self.transformer = transformer  
  
    def process(self, record):  
        return self.transformer.transform(record)
```

Use Strategy when:

the algorithm varies but the workflow remains similar.

Examples:

- parsing formats
- validation rules

- partition strategies
- serialization
- retry strategies

22. Design Pattern: Factory

Useful when configuration determines which implementation to construct.

```
def create_source(config):  
    if config.type == "s3":  
        return S3Source(config)  
  
    if config.type == "api":  
        return ApiSource(config)  
  
    raise ValueError(  
        f"Unsupported source: {config.type}"  
    )
```

Then:

```
source = create_source(config)  
pipeline = Pipeline(source)
```

The creation logic is separated from execution logic.

Very useful for config-driven frameworks.

23. Strategy + Factory Together

This is a common platform architecture.

```
Configuration
  |
  v
Factory
  |
  v
Source implementation
  |
  v
Pipeline
```

Example configuration:

```
source:
  type: s3

parser:
  type: avro
```

Factory creates:

```
S3Source
AvroParser
```

Then injects them into:

```
Pipeline
```

This is a realistic pattern for scalable ingestion frameworks.

24. Adapter Pattern

Useful when external libraries have incompatible APIs.

Imagine one source exposes:

```
client.fetch_objects()
```

but your platform expects:

```
source.read()
```

Adapter:

```
class VendorSourceAdapter:

    def __init__(self, client):
        self.client = client

    def read(self):
        return self.client.fetch_objects()
```

Now the platform does not care about the vendor-specific API.

Typical DE uses:

- cloud SDK wrappers
- database clients
- Kafka clients
- external REST APIs
- legacy systems

25. Template Method Pattern

Useful when the overall pipeline flow is fixed but individual steps differ.

Conceptually:

```
class Pipeline:

    def run(self):
        data = self.extract()
        data = self.transform(data)
        self.validate(data)
        self.load(data)
```

Subclasses customize steps.

This can be useful, but I would generally prefer **composition + Strategy** for modern data platform code because deep inheritance becomes difficult to maintain.

26. Builder Pattern

Useful when constructing complicated configuration objects.

Instead of:

```
Pipeline(  
    source,  
    parser,  
    validator,  
    retries,  
    metrics,  
    writer,  
    ...  
)
```

a builder can progressively construct the pipeline.

```
pipeline = (  
    PipelineBuilder()  
    .with_source(source)  
    .with_validator(validator)  
    .with_writer(writer)  
    .build()  
)
```

Useful occasionally, but lower priority for interviews than Strategy, Factory and Adapter.

27. Repository Pattern

Relevant when you want to isolate persistence logic.

```
class CustomerRepository(Protocol):  
    def find(self, customer_id: str):  
        ...
```

Implementation:

```
class PostgresCustomerRepository:  
    def find(self, customer_id):  
        ...
```

Business logic does not need to know SQL/database details.

Useful in platform/backend systems, although less relevant for pure Spark pipelines.

28. Observer / Event Pattern

Conceptually:

```
Pipeline completed
  |
  +----> Metrics
  |
  +----> Audit
  |
  +----> Notification
```

Instead of tightly coupling every side effect into pipeline logic.

Useful for:

- event-based architectures
- instrumentation
- audit events
- monitoring

29. Functional Programming Concepts

Python is not purely functional, but functional concepts are useful for data transformations.

Pure-ish transformation:

```
def normalize(record):
    return {
        **record,
        "email": record["email"].lower(),
    }
```

Input is not mutated.

This is preferable to:

```
def normalize(record):
    record["email"] = record["email"].lower()
    return record
```

Why?

Mutation can cause surprising side effects.

For pipelines, thinking in terms of:

```
Input → Transformation → Output
```

often leads to simpler code.

30. Mutable Default Arguments

Classic Python trap.

Never:

```
def add_event(event, events=[]):  
    events.append(event)  
    return events
```

The same list survives between calls.

Use:

```
def add_event(event, events=None):  
  
    if events is None:  
        events = []  
  
    events.append(event)  
  
    return events
```

This is a common interview question.

31. Shallow Copy vs Deep Copy

Consider:

```
original = {  
    "customer": {  
        "id": 1
```

```
}  
}  
  
copy = original.copy()
```

The nested dictionary is still shared.

```
copy["customer"]["id"] = 2
```

also affects the nested object referenced by `original`.

For a fully independent copy:

```
from copy import deepcopy  
  
copy = deepcopy(original)
```

But deep copying large datasets can be expensive.

For data engineering, minimizing unnecessary copies matters.

32. Equality vs Identity

Know the distinction.

```
a == b
```

compares values.

```
a is b
```

compares object identity.

Use:

```
if result is None:
```

not:

```
if result == None:
```

33. Hashability

Dictionary keys and set elements must be hashable.

Works:

```
key = ("DE", "purchase")
```

Doesn't:

```
key = ["DE", "purchase"]
```

because lists are mutable and therefore unhashable.

This matters when designing aggregation keys.

34. Closures

A nested function can capture external state.

```
def create_validator(required_fields):  
  
    def validate(record):  
        return all(  
            field in record  
            for field in required_fields  
        )  
  
    return validate
```

Usage:

```
validate_order = create_validator(  
    ["order_id", "amount"]  
)
```

Useful for configurable behavior, although don't make architecture unnecessarily clever.

35. Concurrency Basics

Senior platform engineers should understand the distinction.

Threading

Useful mainly for I/O-bound workloads.

```
API calls
database requests
network operations
```

Multiprocessing

Useful for CPU-bound workloads.

```
heavy Python computation
compression
CPU-intensive parsing
```

Asyncio

Useful when coordinating many concurrent I/O operations.

```
async def fetch(client, url):
    return await client.get(url)
```

36. The GIL

High-level understanding is enough.

CPython's Global Interpreter Lock historically means multiple Python threads do not simply execute CPU-heavy Python bytecode fully in parallel.

Therefore:

```
I/O bound
→ threads / asyncio may help
```

CPU bound
→ multiprocessing / native/vectorized libraries may help

Do not make the simplistic statement:

"Python threads are useless because of the GIL."

They can be very useful for I/O.

37. `async` / `await`

Example conceptual API ingestion:

```
async def fetch_partition(client, partition):  
    return await client.fetch(partition)
```

Multiple requests:

```
results = await asyncio.gather(  
    fetch_partition(client, "DE"),  
    fetch_partition(client, "FR"),  
    fetch_partition(client, "US"),  
)
```

Good for many independent network calls.

But:

Async code is not automatically faster.

It helps when concurrency matches the workload.

38. Logging

Production code should not rely on:

```
print(...)
```

Use structured logging.

```
logger.info(  
    "Processing partition %s",  
    partition,  
)
```

Prefer logs containing operational context:

```
pipeline  
dataset  
partition  
run_id  
record_count  
duration  
status
```

For example:

```
pipeline=orders_ingestion  
partition=2026-08-25  
records=1253043  
duration=43.2  
status=success
```

39. Testing

A Senior Python engineer should distinguish several levels.

```
Unit Test  
  ↓  
Integration Test  
  ↓  
Contract Test  
  ↓  
End-to-End Test
```

Unit test

```
def test_normalize_email():  
    result = normalize({  
        "email": "USER@TEST.COM"
```

```
    })  
  
    assert result["email"] == "user@test.com"
```

Integration test

Actually interact with:

- database
- object storage
- Kafka
- external service emulator

Data-quality test

Verify:

```
schema  
uniqueness  
nullability  
ranges  
referential integrity  
freshness
```

40. Mocking

Useful for external dependencies.

Instead of calling real S3:

```
class FakeSource:  
  
    def read(self):  
        return [  
            {"id": 1},  
            {"id": 2},  
        ]
```

Then:

```
pipeline = Pipeline(FakeSource())
```

This is one of the reasons dependency injection is so valuable.

41. Idempotency

Not purely a Python concept, but essential to platform engineering.

Suppose the same event arrives twice:

```
event_id = 123
event_id = 123
```

Your processing should ideally avoid producing duplicate business effects.

Typical pattern:

```
if event_id in processed_ids:
    return

process(event)
processed_ids.add(event_id)
```

Real production implementations might use:

- database constraints
- transactional writes
- checkpoint state
- deduplication tables
- event keys

42. Retry Design

Avoid:

```
while True:
    try:
        call_api()
    except Exception:
        continue
```

A production retry strategy should consider:

```
maximum attempts
exponential backoff
```

```
jitter
retryable exceptions
timeout
idempotency
```

Conceptually:

```
1 sec
2 sec
4 sec
8 sec
```

Not every failure is retryable.

For example:

```
HTTP 503
→ retry

invalid schema
→ probably don't retry
```

43. Memory and Performance Thinking

Before optimizing, ask:

```
Where is the bottleneck?
```

Potential problems:

```
CPU
memory
network
disk I/O
serialization
database
algorithm complexity
```

Example:

```
user_ids = [user["id"] for user in users]

if event["user_id"] in user_ids:
```

Membership is $O(n)$.

Better:

```
user_ids = {
    user["id"]
    for user in users
}
```

Membership becomes approximately $O(1)$.

That difference matters at scale.

44. Profiling Before Optimizing

Senior answer:

"I would measure before optimizing."

Useful tools/concepts include:

```
cProfile
timeit
memory profiling
application metrics
```

Don't immediately rewrite everything with multiprocessing.

First determine where time is actually being spent.

45. Serialization

Data platforms frequently move data between systems.

Know conceptually:

```
JSON
Avro
```

```
Parquet
Protobuf
```

Python might parse JSON easily:

```
import json

record = json.loads(payload)
```

But production trade-offs include:

```
schema
size
serialization speed
compatibility
columnar access
schema evolution
```

46. Configuration Design

Avoid spreading constants throughout code.

Bad:

```
bucket = "production-orders-eu"
```

everywhere.

Better:

```
@dataclass(frozen=True)
class SourceConfig:
    bucket: str
    prefix: str
```

Configuration should ideally be separated from processing logic.

This allows:

```
same code
+
```

```
different configuration
=
different pipeline
```

A very important platform-engineering principle.

47. Package and Module Design

A realistic small project might look like:

```
data_platform/
├── sources/
│   ├── base.py
│   ├── s3.py
│   └── api.py
├── parsers/
│   ├── json.py
│   └── avro.py
├── validators/
│   └── schema.py
├── writers/
│   └── warehouse.py
├── pipeline.py
├── config.py
└── tests/
```

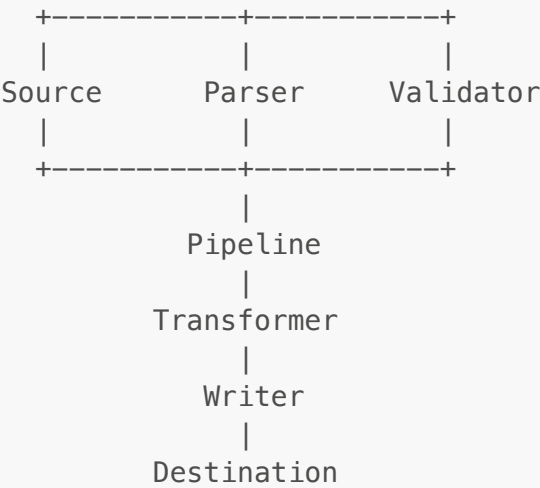
The exact folders do not matter.

The important thing is **clear responsibility boundaries**.

48. A Good Data Platform Architecture in Python

A good mental model for interviews is:

```
Configuration
|
v
Factory
|
```



Cross-cutting concerns:

- Logging
Metrics
Retry
Tracing
Error handling
Configuration
Testing

This is much closer to Senior Data Platform Engineering than simply writing a giant `process_data()` function.

49. The Most Important Design Patterns for You

Don't try to memorize all 23 GoF patterns.

For Senior Data/Data Platform interviews, I would prioritize:

Pattern	Priority	Typical use
Strategy	★★★★★	Change parsing/validation/processing behavior
Factory	★★★★★	Build implementations from configuration
Adapter	★★★★★	Normalize external system interfaces
Dependency Injection	★★★★★	Testable, decoupled architecture
Repository	★★★	Separate persistence
Template Method	★★★	Shared pipeline workflow
Observer/Event	★★★	Metrics/events/notifications
Builder	★★	Complex object construction

Pattern	Priority	Typical use
Singleton	★	Usually avoid

One thing worth remembering:

Design patterns are not goals. They are names for recurring solutions to recurring design problems.

If the code is simple, keep it simple.

50. What “Senior-Level Python” Actually Looks Like

It is not:

```
more classes
more decorators
more abstractions
more design patterns
```

Senior Python is closer to:

```
Correct abstraction
      +
Simple implementation
      +
Explicit contracts
      +
Testability
      +
Failure handling
      +
Operational visibility
```

For example, this can actually be better:

```
def transform(record):
    ...
```

than introducing:

```
AbstractTransformationFactoryProviderManager
```

for a single transformation.

A Senior Engineer should know **when not to abstract**.

51. Interview Mental Model

For a pair-programming problem, think in this order:

1. What is the input?
2. What is the output?
3. What is the key?
4. What state do I need?
5. What is the simplest algorithm?
6. What are the edge cases?
7. What is the complexity?
8. How would this change at production scale?

Only after the basic solution works should you discuss:

```
classes
interfaces
retry
streaming
parallelism
observability
distributed systems
```

52. Recommended Study Priority

For your upcoming Senior DE / Data Platform coding interviews, I would divide these topics into three levels.

Must be automatic

```
list / dict / set
defaultdict / Counter
sorting
functions
comprehensions
exceptions
typing
dataclass
iterator / generator
```

```
basic complexity  
unit testing
```

Must understand deeply

```
OOP  
composition  
ABC  
Protocol  
dependency injection  
SOLID  
Strategy  
Factory  
Adapter  
context managers  
decorators  
mocking  
memory behaviour  
idempotency  
retry patterns
```

Must understand conceptually

```
asyncio  
threading  
multiprocessing  
GIL  
Repository  
Observer  
Builder  
profiling  
serialization  
packaging
```

53. One Architecture Worth Practising From Scratch

If you want one exercise that covers perhaps **70% of this guide**, build this:

Configurable Event Ingestion Framework

Requirements:

```
Read events from different sources  
↓  
Parse different formats
```

↓
Validate records
↓
Deduplicate events
↓
Transform them
↓
Write output
↓
Produce metrics

Then progressively introduce:

Phase 1
simple functions

Phase 2
dataclasses + typing

Phase 3
Source Protocol

Phase 4
Strategy

Phase 5
Factory

Phase 6
Dependency Injection

Phase 7
custom exceptions

Phase 8
unit tests

Phase 9
generators

Phase 10
retry

Phase 11
concurrency

Phase 12
observability

By the end, you will have touched almost every Python concept that is genuinely useful for a Senior Data Platform Engineer.

Most importantly, **don't build all twelve layers immediately**. Start with the simplest working implementation and evolve it when a new requirement creates a reason for the abstraction. That evolution itself is exactly the kind of engineering reasoning you want to demonstrate during a senior pair-programming interview.